

M3 - Gruppe 8 - A10

Racetrack

Finja Rang
Yeonsu Kang
Merle Hoffmann

1. Idee

Die Grundidee war, angelegt an eine Rennstrecke, ein modulares Spiel zu entwickeln, mit dem man sich einen eigenen Song bauen kann. Je nach Bauteil (Linkskurve, Rechtskurve und Gerade) verändert sich der Ton einen Schritt nach Links oder Rechts im Quintenzirkel. Dadurch entsteht eine kurze Melodie, die immer wieder geloopt wird. Dabei wollen wir erreichen, dass man spielerisch eine einfache Melodie mit Kicks und Snares erstellen kann und dabei gleichzeitig eine haptische Darstellung von Tönen erlebt.

Die physikalischen Streckenblöcke haben dabei jeweils einen ATTiny und bilden eine Chain die vom ESP32 ausgelesen wird. Da jedes Streckenteil einen ATTiny mit einzigartiger Erkennungs-ID besitzt, können wir auch ein Kick- und Snare-Pattern pro Modul durch Widerstände und Jumper codieren. Diese kann man während des Loops auch immer umstecken, da wir durch eine I2C-Verbindung mit dem ESP32 diese immer neu ablesen. Somit entsteht zusätzlich ein weiteres Element in der Musik, welches eher an einen Synthesizer anlehnt. Zusätzlich haben wir noch ein 3D-gedrucktes Pedal mit einem Potentiometer, mit dem man durch Drücken dynamisch den BPM der Musik schneller werden lassen kann.

Auch haben unsere Streckenteile jeweils zwei eingebaute Neopixel, sodass man durch Leuchten in den Farben Rot und Grün erkennen kann, wo man sich im Loop und im Abspielen der Töne gerade befindet (das aktuelle Teil leuchtet Grün, während alle anderen Teile rot leuchten).

Der PyBadge dient hauptsächlich als reine UI. Dabei gibt es das Start- und Stopp-Menü sowie einen Warnbildschirm, falls die Strecke nicht erkannt wird. Zusätzlich kann man auch die Startnote am Anfang über die hoch und runter Knöpfe auswählen. Während der Track läuft, gibt der PyBadge eine visuelle Darstellung eines fahrenden Autos aus mit dem aktuell abspielenden Ton, um die Tonerkennung zu vereinfachen.

2. Benötigte Komponenten

- PyBadge
 - GRD Kabel
 - RX Kabel
 - TX Kabel
 - Batterie oder Stromanschluss
 - 3d-Gehäuse

- ESP32
- Arduino UNO zum Attiny flashen
- 6x Jumperkabel zum ATtiny flashen
- Laptop o.Ä. mit USB Anschluss
- Bluetooth-Lautsprecher
- Potentiometer 25k
- 3.3V Netzgerät
- 2,54 mm Stiftleisten
 - diese haben wir in 100x 2er Pins getrennt und dann oben mit Lötzinn verbunden (als homemade Jumper)
 - dann wurden sie mit Heißkleber in die 3D-gedruckten Pylonen geklebt

Wir haben 13 Streckenteile (3 Links- 3 Rechtskurven und 7 Geraden), die alle die gleichen Komponenten haben. Also ist hier ein Beispiel eines Streckenteils und dieses dann 13 mal:

- 8-Bit-ATtiny 85
- Lochrasterplatine(-n) (Gerade: ca. 10 x 4 cm, Kurven: 2x ca. 5 x 4 cm)
- IC-Sockel
- JST - Buchsengehäuse, 1x6-polig - XH
- JST - Stiftleiste, gerade, 1x6-polig - XH
- 6x JST - Crimpkontakt, Buchse - XH
- 1x 100 µF Elektrolyt-Kondensator
- 2x 10k Widerstände
- 2x 1k Widerstände
- 2x 2k Widerstände (2x1k Widerstände)
- 2x 4k Widerstände (1x 1,8k und 1x 2,2k Widerstände)
- 2x 8k Widerstände (1x 6,8k und 1x 1,2k Widerstände)
- 8x 2,0 x 3,0 mm Scheibenmagnete
- 8x 2er Buchsen
 - + weitere 8x 2er Buchsen
 - diese wurden aufeinander gesteckt und mit Schrumpfschläuchen verbunden damit sie lang genug sind um oben aus dem Gehäuse herausragen zu können
- Schrumpfschläuche
- Gnd Kabel (mehrere Stücke)
- Vcc Kabel (mehrere Stücke)
- 2x Neopixel
- Kabel für SDA (mehrere Stücke)
- Kabel für SCL (mehrere Stücke)
- Kabel für Neo-Data (mehrere Stücke)
- Kabel für Chain-Data (mehrere Stücke)
- Sekundenkleber
- Heißkleber
- 3D-Druck Gehäuse

3. Verkabelung

- UART2 zwischen ESP32 und PyBadge
 - GPIO16=RXD2
 - GPIO17=TXD2
- I2C-Bus (Master ESP32) zu dem ersten angesteckten ATTiny (Adressbereich 0x10–0x1F) durch Stecker
 - GPIO21=SDA
 - GPIO22=SCL
- Chain-Signal zwischen ESP32 und dem ersten angesteckten ATTiny
 - GPIO4=PB4 (Chain-In)
- NeoPixel-Datachain zwischen ESP32 und dem ersten angesteckten ATTiny
 - GPIO5=Neopixel Data-In
- ESP32 und Potentiometer (ADC)
 - GPIO34 = Data von Potentiometer
 - ESP 3.3V an Potentiometer
- ESP GND
 - An PyBadge GND
 - An ATTiny GND
 - An Neopixel GND
 - An Potentiometer GND
- 3.3V Netzgerät
 - Vcc an ATTiny VCC
 - VCC an Neopixel VCC
 - GND and ESP32 GND
- ATTiny Pins:
 - PB0 = I2C SDA (Pin 5)
 - PB1 = Chain-Out (zum nächsten Chain-In) (Pin 6)
 - PB2 = I2C SCL (Pin 7)
 - PB3 = Pin_Kick, ADC3 (Pin 2)
 - PB4 = Chain-In (Pin 3)
 - PB5 = Pin_Snare, ADC0 (Pin 1)
 - GND (Pin 4)
 - VCC (Pin 8)
- Kondensator am Anfang des Streckenteils bei GND und VCC Verbindung

4. Installationsanleitung

- ArduinoIDE herunterladen
- Arduino mit Arduino ISP-Sketch flashen (unter File → Examples → ArduinoISP)

- ATTiny Module flashen:
 - ATTiny mit einem Arduino Uno flashen
 - Steckverbindung

Arduino Uno	ATTiny85
Pin 10	Pin1 (PB5)
Pin 11	Pin 5 (PB0)
Pin 12	Pin 6 (PB1)
Pin 13	Pin 7 (PB2)
5V	Pin 8 (VCC)
GND	Pin 4 (GND)

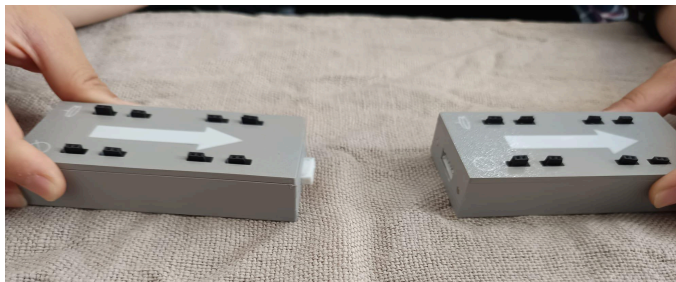
- 10 µF Kondensator zwischen RESET und GND des Arduino Uno
 - ATTiny Board runterladen (<https://github.com/damellis/attiny>)
 - Einstellung von Arduino ISP
 1. Clock Internal 8MHz
 2. Programmer: Arduino als ISP
 3. Processor: ATTtiny 85 auswählen
 - Burn Bootloader
 - Upload Code über Upload using Programmer
 - Fuses setzen (lfuse=0xE2, hfuse=0x5F)
 1. über Arduino IDE oder
 2. Kommandozeile:


```
avrdude -c stk500v1 -p attiny85 -P /dev/ttyUSB0 -b 19200 -U lfuse:w:0xE2:m -U hfuse:w:0x5F:m
```
 - Bei jedem Neuen Modul den Adressbereich 0x10–0x1F umändern und die Richtungen 0(Gerade),1(Links),2(Rchts) anpassen
- ESP32 Modul flashen
 - ESP32 Board runterladen ([GitHub - espressif/arduino-esp32: Arduino core for the ESP32 family of SoCs · GitHub](#))
 - ESP32 Dev Modul auswählen
 - Benötigte Libraries
 1. BluetoothA2DPSource (<https://github.com/pschatzmann/ESP32-A2DP>)
 2. Adafruit_NeoPixel ([GitHub - adafruit/Adafruit_NeoPixel: Arduino library for controlling single-wire LED pixels \(NeoPixel, WS2812, etc.\) · GitHub](#))

3. Wire (USIWire) ([GitHub - puuu/USIWire: USI based TWI/I2C library for Arduino · GitHub](#))
4. functionGenerator ([GitHub - RobTillaart/FunctionGenerator: Arduino library to generate wave forms \(numeric\) for a DAC · GitHub](#))
 - Bluetooth-Lautsprecher-Name im Code eintragen und koppeln
 - Code auf ESP32 Dev Modul flashen
- Pybadge
 - CircuitPython-Firmware installieren
 - Benötigte Libraries (in den "lib" Ordner auf dem CIRCUITPY-Laufwerk kopieren)
 1. adafruit_pybadger (https://github.com/adafruit/Adafruit_CircuitPython_PyBadger)
 2. adafruit_imageload (https://github.com/adafruit/Adafruit_CircuitPython_ImageLoad)
 3. adafruit_display_text (https://github.com/adafruit/Adafruit_CircuitPython_Display_Text)
 - [code.py](#) auf CIRCUITPY-Laufwerk kopieren
 - Bilddateien (bg.bmp, porsche911.bmp, porsche911_2.bmp) auf CIRCUITPY-Laufwerk kopieren
- Hardware miteinander verbinden (UART, Module in Ketten anschließen)
- Stromversorgungen anschließen
- Einschalten vom PyBadge und starten
- Falls nötig: Serial Monitor prüfen (Baud 115200)

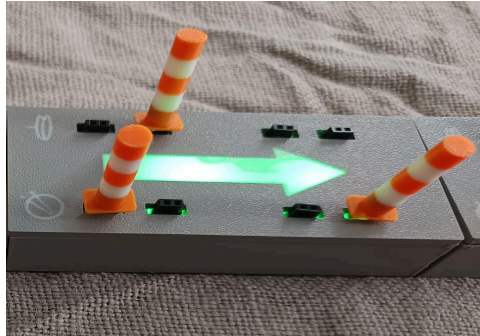
5. Ausführliche Spielanleitung

- Kurz warten bis der Lautsprecher mit dem ESP32 verbunden ist
- Spielende verbinden physische Streckenmodule (L/R/G)



- Die Spielenden starten PyBadge und wählen darüber die Startnote aus (hoch/runter) und drücken START
- Ablauf PyBagde:

1. Boot-Screen
 2. Menü (Startnote)
 3. Check_Start ob Module angeschlossen sind
 4. Warnbildschirm falls keine Strecke erkannt oder da ist
 5. Race-Countdown durch Rote LEDs am PyBadge
 6. Spielbildschirm mit Noten/Racing-Animation
- Interaktive Elemente: Kick-/Snare-Pattern kann pro Modul durch das Ein- und Ausstecken der Jumper während des Spiels verändert werden. Die Jumper stehen in Form von Pylonen zur Verfügung.



- Pedal kann von Spielenden runter gedrückt werden (mit Füßen oder Händen) um BPM zu erhöhen
- Um das Spiel zu beenden oder eine neue Strecke zu bauen, drückt man die Taste SELECT.
- Erlaubt/nicht erlaubt:
 - Die maximale Tracklänge ist theoretisch 16 Stück, auch wenn wir nur 13 zur Verfügung haben.
 - BPM-Grenzen liegen bei 70-200
 - Das Pedal ist stabil, sollte aber trotzdem nicht mit voller Kraft runtergedrückt werden.
 - Die Stecker lassen sich manchmal etwas schwer verbinden/trennen, also darf man da auch ruhig mit etwas mehr (nicht voller) Kraft hantieren.

6. Kurzanleitung: separat

7. Bekannte Probleme & Fixes

- Die Steckverbindungen sind etwas fragil. Früher hatten wir nur Pins und Buchsen benutzt, die noch fragiler waren und jetzt benutzen wir JST Stecker, die sind etwas besser. Falls man die Rennstrecke aber nachbauen möchte, würden wir vorschlagen, die jeweiligen Steckverbindungen mit magnetischen Pogo-Pin-Verbindungen nachzubauen. Diese sind etwas kostspieliger aber wären definitiv besser für eine stabile und einfache Verbindung.